

UNIVERSIDAD AUTÓNOMA DE SINALOA
UNIDAD ACADÉMICA FACULTAD DE INFORMÁTICA
LIC. EN INFORMÁTICA



MATERIA:

DESARROLLO WEB DEL LADO DEL SERVIDOR

MEDIADOR:

JOSÉ MANUEL CAZARES ALDERETE

ACTIVIDAD:

SEGURIDAD BASADA EN TOKENS (JWT)

ALUMNA:

NAVARRETE REYES MARITZA GUADALUPE

GRADO Y GRUPO: 2-1 **TURNO:** MATUTINO

Crear el siguiente CRUD (ya lo deberían de tener por que se dejó como actividad la semana pasada), y proteger todas las rutas mediante la seguridad basada en tokens con JWT.

Cientes

- id
- nombre
- correo
- telefono
- direccion

app.js

```
const express = require('express')
const bodyParser = require('body-parser')
const jwt = require('jsonwebtoken')
const clientes = require('./clientes')
const { Op } = require('sequelize') // para utilizar operadores de sequelize

const app = express()
const puerto = 3000

const secretKey = 'secret'

app.use(bodyParser.json())

app.listen(puerto, () => {
  console.log('servicio iniciado')
})

// Si el usuario y contraseña son correctos, genera y retorna un token.
// Si no, retorna un error 404 (Not Found).

app.post('/login', (req, res) => {
  const { usuario, password } = req.body;
  if (usuario == 'admin' && password == '123') {
    const token = jwt.sign({ usuario }, secretKey, { expiresIn: '1h' });
    // utilizar JWT
    res.send(token)
  }
})
```

```

    } else {
      res.status(404);
    }
  })

// Si el token es válido, permite el acceso al siguiente middleware o ruta.
// Si no es válido, retorna un error 401 (Unauthorized).
function verificarToken(req, res, next) {
  const header = req.header('Authorization') || '';
  const token = header.split(' ')[1];
  if (!token) {
    res.status(401).json({mensaje: 'Token no proporcionado'});
  } else {
    try {
      const payload = jwt.verify(token, secretKey);
      next();
    } catch {
      res.status(401).json({mensaje: 'Token incorrecto'});
    }
  }
}

// Endpoint para agregar un nuevo cliente.
// Requiere autenticación mediante token.

app.post('/agregar', verificarToken, async (req, res) => {
  const { nombre, correo, telefono, direccion } = req.body;
  const data = await clientes.create({
    nombre, correo, telefono, direccion
  });
  res.send(data);
});

// Endpoint para obtener todos los clientes.
// Requiere autenticación mediante token.

app.get('/buscar', verificarToken, async (req, res) => {
  const data = await clientes.findAll();
  res.send(data);
});

// Endpoint para actualizar un cliente por ID.
// Requiere autenticación mediante token.

```

```
app.put('/actualizar/:id', verificarToken, async (req, res) => {
  const { nombre, correo, telefono, direccion } = req.body;
  const { id } = req.params;
  const data = await clientes.update({
    nombre, correo, telefono, direccion
  }, {
    where: {
      id
    }
  })
  res.send(data);
});

// Endpoint para eliminar un cliente por ID.
// utilizando autenticación mediante token.

app.delete('/eliminar/:id', verificarToken, async (req, res) => {
  const { id } = req.params;
  const data = await clientes.destroy({
    where: {
      id
    }
  });
  res.send(data);
});
```